Rules Decide, Language Models Narrate: Fail-Closed Numeric Grounding of LLM Narration in a Flood-Response Pipeline

Nandhakumar T
Department of Information Technology
Sona College of Technology
Salem, Tamil Nadu, India
nandhakumartm93@gmail.com

Balakarthikeyan B S
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
balasan2626@gmail.com

Gowthaman J
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
gowthamanjanadhran@gmail.com

Abhishek S
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
abisheksengodan18@gmail.com

***Abstract*—Large language models (LLMs) are proposed as the voice of emergency-management software, but they can state numbers that the data do not contain. We measure this risk in a deployed flood-response prototype in which deterministic rules choose every action and an LLM only verbalises the state as operator messages, behind a fail-closed numeric guard that replaces a reply containing an unsupported number by a fixed template. In 300 live replies of two open-weight models, 36.3% contained an unsupported number, almost all of them invented arrival times; the deployed guard caught 89.0% of these replies. We define a grounded reply and show that a narrator which writes placeholders instead of numbers is grounded by construction within a stated lexicon; tested live, it left a digit or number word in 2.1% of replies, which then fell back to the template. A revised guard, frozen by hash before a third model family was queried, met a different error mix on that model: 57.6% of its replies were unsupported, mostly invented head-counts and misquoted facts, and the revised guard was not significantly better than the deployed one there. An error taxonomy and a component ablation show which rule catches which error. Qualitative over-claims such as "flood confirmed", partly seeded by our own prompt and template, appeared in 14.3% of replies and pass every numeric check. A composed check of the decision path frames the safety argument. All rates are specific to our prompt, models and states.**

***Index Terms*—large language models, hallucination, data-to-text generation, fail-safe design, runtime verification, flood response**

I. Introduction

During a flood, an emergency operations centre runs on short spoken messages: the river is at this level, send a team there. Software that supports the centre is increasingly asked to *speak*, and LLMs make this easy. They also produce confident statements that their input does not support [1]. On an operations net a wrong number, or a wrong claim that a flood has started, can trigger or cancel an evacuation.

Agentic systems let the model plan and call tools [2], [3]. For a safety-critical setting we take the opposite position: decisions are made by small deterministic rules that can be checked exhaustively, and the LLM only verbalises their state as operator messages. This follows the view that the level of automation should be chosen per function and per risk [4] and that safety is a property of the control structure rather than of one component [5]. The design is close to a classical alerting pipeline, in which a rule fills a structured message such as a Common Alerting Protocol (CAP) alert [6]; what is new is a free-text narrator inside it. We ask (Q1) whether the verified decision governs actuation and (Q2) how often the narrator misstates the state, and which check or design prevents it. Q1 is the precondition of the safety argument; Q2 is our subject.

We answer both on an open flood decision-support prototype for the Delhi Yamuna corridor. The contributions are measurements on the deployed code, not a new detector:

1) A definition of a grounded reply and a proposition that a placeholder-only ("slots-only") narrator is grounded by construction, with its scope, tested live against the production prompt.
2) A measurement of numeric grounding on 300 live replies of two open-weight LLMs, with an error taxonomy over all 999 stored replies, state-clustered intervals and a comparison of the deployed guard, a typed guard and an LLM judge.
3) A held-out test of a revised guard on a third model family, frozen by hash before its replies were read, with a component ablation, and the template-fallback rate as the monitoring signal.
4) Evidence that part of the over-claiming came from our own prompt and template, which no numeric check can

Algorithm 1 One pipeline tick (after the fix)

read observations and the operator override flag o
$p_{\mathrm{RF}}, p_{\mathrm{XGB}}, p \leftarrow$ classifiers; $H \leftarrow$ (2)
$(h_d, h_m, h_s) \leftarrow$ health checks; $a \leftarrow \pi(p, \mathbf{h}, o)$ by (1)
if H **and not** o **then** $a \leftarrow$ REVIEW and log the hold reason
if $a \neq$ MONITOR **then** rank roads, shelters, routes
if $a \in \{\text{AUTO}, \text{OVERRIDE}\}$ **and** band(p) = auto **then** dispatch
$T \leftarrow$ LLM turns for band(p); **if not** $\Gamma(T)$ **then** $T \leftarrow$ template
broadcast state, decision and T

see.

A companion paper on the whole decision chain reuses the dispatch result and the development-set guard results; this paper reports the full narration study.

II. Related Work

Agents and guardrails. LLM agents are surveyed in [7]. Generated text departs from its source in many ways [1]. Retrieval grounds generation in documents [8]; post-hoc checkers decompose text into atomic claims [9], sample the model several times [10], let the model verify itself [11] or ask another model to judge [12]. NeMo Guardrails programs dialogue rails and checks facts with a further model call [13], the setting of our LLM-judge baseline; runtime verification checks outputs against expert specifications [14], and runtime enforcement blocks unsafe agent actions [15].

Data-to-text faithfulness. Our narrator is a data-to-text generator. Neural data-to-text systems were shown early to state numbers and facts absent from their input records [16]; a gold-standard error annotation separates number, name, word and context errors [17], and open LLMs still produce such errors on structured data [18]. Delexicalisation, in which the generator writes slot tokens that a program later fills, has long been used in dialogue generation to keep values exact [19]; our slots-only narrator is that idea applied to an LLM, as a safety device rather than a data-sparsity device.

Safe automation. Shields block unsafe actions of learned controllers [20], and ensembles estimate uncertainty [21].

Position. The checks themselves are simple. What we add is an empirical measurement inside a deployed safety pipeline: a guard frozen by hash and tested on a model family it never saw, the fallback rate as the signal that a guard has started to fail, a formal scope for the placeholder design, and the finding that the system's own prompt and fallback template produced over-claims.

III. Architecture

Fig. 1 shows the system. A FastAPI backend runs one pipeline per tick, in live mode (Open-Meteo weather and river data) or in simulation mode (a digital twin of the corridor). Of its eleven components, which the code calls agents, only the policy (1) decides, and dispatch executes only what it allowed. Every event goes to a MongoDB log. Algorithm 1 gives one tick; narration runs last, after the decision is logged, so a narration failure can delay a message but cannot change a decision.

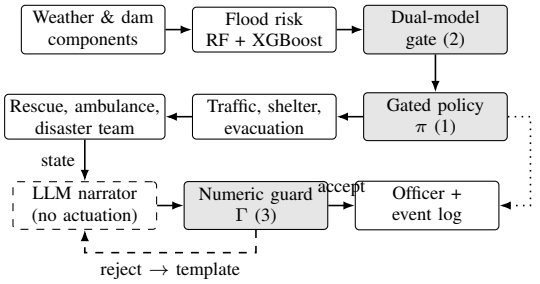


Fig. 1. Pipeline. Shaded blocks can only hold, downgrade or reject; none can trigger an action. The LLM (dashed) receives the shared state and has no actuation rights. The dotted link carries every policy decision and its reason to the officer.

A. Decision Policy and Disagreement Gate

The flood-risk component receives a flood probability p and three health flags: h_d (weather and river data available), h_m (model loaded and returned a value) and h_s (database connected). An operator can set an override o, and a flag A enables automation. With $\tau_m = 0.50$ and $\tau_a = 0.60$, served from one configuration module,

$$\pi(p, \mathbf{h}, o) = \begin{cases} \text{UNAVAILABLE} & p \text{ undefined} \\ \text{OVERRIDE} & o \\ \text{MONITOR} & p < \tau_m \\ \text{AUTO} & p \geq \tau_a \wedge h_d \wedge h_m \wedge h_s \wedge A \\ \text{REVIEW} & \text{otherwise.} \end{cases} \tag{1}$$

A Random Forest [22] and an XGBoost model [23], the latter supplying p, were trained on the INDOFLOODS catalogue (4,548 events at 155 gauges) to predict whether an event is severe (held-out ROC-AUC 0.818 and 0.806). Automatic dispatch is held when

$$H = \big[\, |p_{\mathrm{RF}} - p_{\mathrm{XGB}}| \geq \delta \,\big] \\ \vee \ \big[\min(p_{\mathrm{RF}}, p_{\mathrm{XGB}}) < \tau_a \leq \max(p_{\mathrm{RF}}, p_{\mathrm{XGB}}) \big], \tag{2}$$

with $\delta = 0.15$; the second, *straddle* term flags models on opposite sides of the threshold.

B. LLM Narration and the Numeric Guards

The band follows p (calm below 0.40, watch below 0.50, administrator review below 0.60, automated response above) and sets who speaks and the tone; the action is still set by π. The prompt lists the current facts (probabilities, rainfall, river level, barrage stage, time and place), states the band's rule and tells the model to use only those numbers. The rule of the automated band reads "Above 60%: flood occurred", and the template that replaces a rejected reply begins "Flood has occurred"; Sec. V-G returns to this wording.

Let $\mathcal{N}(s)$ be the decimal numbers in a string s and F the prompt text up to the recent-history block. With $\tilde{F} = \{f, 100f, f/100 : f \in \mathcal{N}(F)\}$, the deployed guard accepts turns T when

$$\Gamma(T) = \bigwedge_{t \in T} \ \bigwedge_{n \in \mathcal{N}(t)} \ \exists f \in \tilde{F} : \ |n - f| \leq \tfrac{1}{2} 10^{-d(n)}, \tag{3}$$

where $d(n)$ is the number of decimals written in n, so honest rounding passes. The guard is untyped: a number written as a word is invisible to it, and an integer drawn uniformly from $[a, b]$ passes with probability

$$P_{\text{miss}} = |\{\lfloor f \rceil : f \in \tilde{F}\} \cap [a, b]| \,/\, (b - a + 1), \qquad (4)$$

where $\lfloor f \rceil$ is the integer nearest to f (the only integers within the tolerance of (3) for $d(n) = 0$).

Typed guards. Γ_{T} reads facts from the structured state and binds each number to a quantity before checking it: number words become digits, clock times must equal the current time, a number with "%" must match a model probability or a threshold of the band, one with "mm" the rainfall and one with "m" a water level. An unbound number is rejected. A revised version, $\Gamma_{\mathrm{T}}^{(2)}$, adds a list of procedural numbers (reporting intervals, deadlines given as orders), rejects every arrival time and binds a value without a unit to the quantity named just before it.

Slots-only narrator. Instead of checking numbers, the prompt can forbid them: the model writes no digit or number word and refers to facts by placeholders ({probability}, {rain_mm}, {river_m}, {barrage_m}, {time}, {threshold}, {places}), which the program fills from the state. A reply whose raw text contains a digit, a word from a fixed number-word lexicon $\mathcal{W}$ or an unknown placeholder is replaced by the template.

Definition 1 (Grounded reply): Let the state σ assign a value v_q to each quantity q it holds. A *numeric mention* is a number n in the reply, in digits or words, together with the quantity q it describes. A reply is *grounded* in σ if every mention is procedural, quotes a threshold of the current band, or has q in σ and $|n - v_q| \le \frac{1}{2}10^{-d(n)}$. It is *value-grounded* if every number it contains equals some state value or threshold, whatever quantity it is attached to.

Proposition 1 (Slots-only render): If the raw text of a slots-only reply contains no digit, no word of $\mathcal{W}$ and no unknown placeholder, and no place name contains a digit, then the rendered reply is value-grounded. It is grounded if, in addition, the model wrote each placeholder where its quantity is meant.

Proof: Every digit of the rendered text was inserted by the renderer, which writes only state values and band thresholds into known placeholders; the raw text contributes none, and no number word from $\mathcal{W}$. ∎

The guarantee has two stated gaps: number words outside $\mathcal{W}$ ("eleven", "a dozen") and a placeholder put where another quantity is meant. Neither is a guard failure in the sense of (3); both are checked empirically in Sec. V-D.

IV. Evaluation Setup

Decision path. We evaluated (1) on $p \in \{0, 0.001, \ldots, 1\}$ plus an undefined value, all eight health combinations and both override states (16,032 cases), against five properties (AUTO only if every check passes and $p \ge \tau_a$; undefined p gives UNAVAILABLE; $p < \tau_m$ without override gives MONITOR; $p \ge \tau_a$ with a failed check gives REVIEW; monotone in p when healthy). We then extracted the dispatch decision into a pure function and checked the composed path (policy, hold, dispatch) for the property that teams are dispatched only when every check passes and the models agree, or an operator overrides, over 32,032 cases.

Lemma 1 (Grid coverage): Assume (i) the extracted policy and dispatch functions read p only through the comparisons $p < 0.40$, $p < 0.50$ and $p \ge 0.60$; (ii) these thresholds are grid points and the code compares against the same binary floating-point constants that the grid produces ($k/1000$ in IEEE 754 double precision); (iii) H enters the dispatch function as a free boolean, enumerated in both values for every p; (iv) health flags and override are booleans. Then the grid check of a property over $\{0, 0.001, \ldots, 1\}$ decides it for every $p \in [0, 1]$.

Proof: By (i), the outputs are constant on each of $[0, 0.40)$, $[0.40, 0.50)$, $[0.50, 0.60)$ and $[0.60, 1]$; by (ii), each interval contains grid points and its end points fall in the right interval; by (iii) and (iv), every other input is enumerated. ∎

The lemma covers the extracted functions. Because H is a free input, the check holds for any hold the gate might output; it does not verify that (2) computes H correctly, and it says nothing about the asynchronous pipeline around the functions.

Live replies. For 150 seeded states (rainfall 5–250 mm in a monsoon month, both model probabilities from the deployed classifiers, river 200.0–208.5 m, barrage 199.0–206.0 m) we sent the production prompt through the Groq API to `openai/gpt-oss-20b` and `openai/gpt-oss-120b` on 10 Oct. 2026 (300 replies, all parsed), without fallback, and stored every verdict. The temperature was 0.85 because that is the value the deployed narrator uses; we did not tune it. Reference labels follow a written protocol. A number is *supported* if it states a state value for the quantity it describes (honest rounding allowed) or quotes a band threshold; *procedural* numbers (reporting intervals, the count of named places, band ranges, deadlines given as orders) are ignored; anything else, including any arrival time, is *unsupported*. A script applies the protocol; replies on which it disagreed with any checker were then read by one author, and 4 labels were changed, all to "procedural". That reading saw the checkers' verdicts, so we also report results under the script's labels alone. Intervals resample states (2,000 replicates), since both models answer the same states.

Prompt designs. For the first 120 states and the same models we ran on 13 Oct. 2026 the production prompt plus one line forbidding arrival times, and the slots-only prompt (233 calls each), plus the unchanged production prompt on the first 50 states (100 calls) to check for drift. These replies were labelled by the script only.

LLM judge. Qwen3 27B (API identifier `qwen/qwen3.8-27b`), a family different from the generators, received the structured facts and the turns at temperature 0 and returned whether every number was supported [12], for 320 injected replies (40 states) and the 300 live replies (10–11 Oct. 2026). 23 live calls failed after retries; the deployed system would send these to the template.

Held-out model family. $\Gamma_{\mathrm{T}}^{(2)}$ was written from the misses on the development replies, frozen and hashed (SHA-256) before

TABLE I
CHECKS ON THE 300 LIVE GPT-OSS REPLIES (REFERENCE LABELS)

Check	n	Recall	False rej.	Calls
Deployed Γ	300	89.0 (83–95)	5.8 (3–9)	0
Typed Γ_{T}	300	93.6 (89–98)	10.5 (6–15)	0
LLM judge, failures rejected	300	99.1 (97–100)	14.1 (9–19)	1
LLM judge, answered only	277	99.0 (97–100)	6.8 (3–11)	1

In %, 95% state-clustered bootstrap intervals in brackets. Recall over unsupported replies, false rejections over supported ones. "Answered only" drops the 23 judge calls that failed. Calls: model calls per check.

replies of `allam-2-7b` were collected for the same 150 states on 10 Oct. 2026. 132 replies parsed; the other 18 calls, all in the automated band, failed or returned invalid JSON. Reading changed 8 labels, again all to "procedural".

Claim check. A short pattern check rejects sentences that assert an observed flood ("flood confirmed", "has breached", "water ingress") unless a hedge ("could", "chance of", "expected") precedes them. It was written on the development replies and frozen by hash before any reply of the prompt-design run was read. Its accuracy has not been measured.

V. RESULTS

A. The Decision Path

The policy satisfied all five properties in every case. The composed check found that the original dispatch step tested the probability band and the hold but never read the action returned by π: when a health check failed while both models agreed on a high probability, π returned REVIEW and teams were still dispatched (2,807 grid cases, one equivalence class). In the prototype's own rainfall sweep, 119 of 604 settings were in that state. After the fix (dispatch requires AUTO or OVERRIDE, and the hold applies in both modes) the check found 0 violations. An end-to-end test would also have caught this integration error; we report it to show that a property check on the composed path finds it systematically, and because the rest of the paper relies on it: once only π can actuate, a wrong narration can mislead an officer but cannot dispatch a team.

B. Numeric Grounding of Live Replies

109 of the 300 replies (36.3%, 95% interval 31–42%) stated at least one unsupported number, and 106 of these 109 were invented arrival times such as "ETA 10 minutes"; the prompt contains no ETA. Table I compares the checks.

The typed guard caught more than the deployed guard but rejected 20 supported replies, mostly for procedural numbers or values without a unit ("barrage 202.26"). Its extra false rejections are clear (difference 2.1 to 8.0 points); its extra recall is weaker evidence, since the exact paired test gives $p = 0.062$ while the clustered interval of the difference is 1.0 to 9.0 points. The LLM judge recognises an invented ETA in any wording, but it costs one model call per message, failed on 23 of 300 calls, and is itself a model whose verdicts cannot be checked exhaustively. Under the script's labels alone (113 unsupported replies instead of 109), the guards' recall was 88.5%, 93.8% and 96.5%.

TABLE II
ERROR TAXONOMY OVER ALL 999 STORED REPLIES

	Replies with error (%)					Passed (%)		
Category	20b	120b	ALLaM	Var.	n	Γ	Γ_{T}	$\Gamma_{\mathrm{T}}^{(2)}$
ETA	32.0	38.7	4.5	4.9	140	14	8	3
Head-count	0.0	0.0	25.0	0.0	33	0	0	0
Duration	2.7	1.3	9.8	0.0	19	11	0	0
Coordinate	0.0	0.0	0.8	0.0	1	0	0	0
Label or level	0.7	0.0	5.3	0.0	8	0	0	0
Misquoted fact	2.7	0.0	37.9	1.9	65	8	8	10
Number word	0.0	0.7	3.0	0.4	7	57	0	0
Other	0.0	0.0	1.5	0.0	2	0	0	0

20b, 120b: 150 replies each; ALLaM: 132; Var.: the 567 prompt-design replies. n: replies with the error. Passed: share of them a guard accepted; $\Gamma_{\mathrm{T}}^{(2)}$ only on the 699 replies not used to design it. A reply can fall in several rows. No rate of rise was flagged.

C. What the Errors Are

Table II classifies every unsupported reply that we stored. Categories are assigned by keyword rules on the context of each number the labelling script flagged (for example, a number followed by "people" or "affected" is a head-count); nobody read the replies to assign them. The two development models erred almost only with arrival times, the held-out model with head-counts, durations and misquoted facts.

The deployed guard passed 14% of replies with an invented ETA and 57% of those with a number word. Eq. (4) explains only part of this. The 82 development replies whose only error was an ETA in digits used 5–30 minutes; over that range, (4) gives a miss probability of 5.2% on average (at most 11.5%) for the facts of each state, so 4.2 of these replies should pass by collision (4.6 with the observed distribution of values, which favours 10 and 15). three did. Of the 12 development replies the deployed guard missed, the other nine lie outside (4): three wrote the ETA in words, five as a clock time and one contained another error. Misquoted facts are the class that every guard still passed (8–10%); a value close to a fact for another quantity passes an untyped check, and the typed guards bind by nearby words, which a long sentence can mislead.

D. Prompt Designs

Table III gives the alternative prompts. Re-run on the same day, the production prompt still produced an unsupported number in 28.0% of replies (20.0–38.0%). One added line forbidding arrival times lowered this to 3.4% (1.3–6.0%) at a cost of 30 tokens per call. The slots-only prompt left a digit or number word in 2.1% of replies, which fell back to the template.

One of the 229 replies that passed the slot check was still flagged by the labelling script. Read against the Proposition, it is not a counterexample. The flagged number is the river level, a state value; the model wrote "{rain_mm} mm rainfall, {river_m} metres river level", which renders a doubled unit, and the script binds a number to rainfall when "rain" occurs just before it. The same reply wrote "Estimated time {time}",

TABLE III
PROMPT DESIGNS, LIVE, FIRST 120 STATES (SCRIPT LABELS)

Prompt	Calls	Unsup.	ETA	Templ.	Claim	Tokens	Words
production prompt*	100	28.0	26.0	26.0	8.0	634	64
+ no arrival times	233	3.4	0.4	2.6	17.6	664	67
slots only	234	1.7	0.4	2.1	5.1	734	57

Unsup., ETA, Templ., Claim in % of replies. Templ.: replaced by the template (failed call or rejected by Γ; for slots, rejected by the slot check). Claim: flagged by the claim check (accuracy not measured). Tokens: prompt + output per call; words per reply. *First 50 states only.

TABLE IV
GUARDS ON DEVELOPMENT REPLIES AND ON A HELD-OUT MODEL FAMILY

	gpt-oss, $n = 300$		ALLaM-2, $n = 132$ (held out)	
Check	Recall	False rej.	Recall	False rej.
Deployed Γ	89.0	5.8	96.1 (91–100)	7.1 (2–15)
Typed Γ_{T}	93.6	10.5	100.0 (100–100)	17.9 (8–29)
Typed $\Gamma_{\mathrm{T}}^{(2)}$ (frozen)	100.0	0.0	98.7 (96–100)	12.5 (4–23)

In %, 95% bootstrap intervals over states. $\Gamma_{\mathrm{T}}^{(2)}$ was frozen by hash before any ALLaM-2 reply was read.

so the current time is presented as an arrival time: value-grounded, but with a placeholder where another quantity is meant, the second gap of the Proposition. A scan of the passed replies found no number word outside $\mathcal{W}$. Whether placeholder replies read as well on a radio net is a question for officers.

E. Injected Errors

On 200 states, each with one grounded reply and seven copies holding one wrong value, passed through the production code with the HTTP client stubbed, the deployed guard rejected 96.0% of wrong digit strings, none of the grounded replies and none of the wrong values written in words. Its 48 misses were collisions: 39 with a genuine fact for another quantity, 8 with a band threshold and 1 with a $\times 100$ copy. The typed guard rejected 98.8% of wrong digit strings and 81.5% of wrong number words. On a subset of 40 states the LLM judge was the weakest check on small probability errors, which change a single digit (it missed 15%).

F. A Held-Out Model Family

Table IV separates the development replies, on which $\Gamma_{\mathrm{T}}^{(2)}$ was built and is perfect by construction, from the frozen test on ALLaM-2. 76 of its 132 replies (57.6%) were unsupported, with the error mix of Table II. The frozen guard caught 98.7% against 96.1% for the deployed guard, a difference of two replies (McNemar $p = 0.62$; clustered interval -2.5 to 8.0 points), and rejected 7 correct replies against 4, all of them procedural labels such as "Team 1" that the development models never wrote. On this set neither guard is shown to be better. Under the script's labels, which count those labels as unsupported, neither guard falsely rejected any held-out reply, which shows how much false-rejection rates depend on the procedural judgement.

TABLE V
ABLATION OF $\Gamma_{\mathrm{T}}^{(2)}$: RECALL / FALSE REJECTIONS (%)

	gpt-oss (dev.)		ALLaM-2		Prompt designs	
Guard	Rec.	FR	Rec.	FR	Rec.	FR
complete $\Gamma_{\mathrm{T}}^{(2)}$	100.0	0.0	98.7	12.5	85.0	0.0
– number words	99.1	0.0	97.4	12.5	80.0	0.0
– clock rule	100.0	0.0	92.1	12.5	85.0	0.0
– unit binding	100.0	0.0	98.7	12.5	85.0	0.0
– ETA ban	85.3	0.0	98.7	12.5	65.0	0.0
– procedural list	100.0	18.8	100.0	16.1	85.0	4.9

"–": component switched off. Labels: reference (dev., ALLaM-2), script (designs).

Table V removes one component of $\Gamma_{\mathrm{T}}^{(2)}$ at a time. The ETA ban carries most of the recall on the development models and on the prompt-design replies (40 unsupported of 567, also unseen when the guard was frozen); the clock rule matters only on the held-out model; the procedural list prevents false rejections and catches nothing; unit binding changed no verdict on any set. On the prompt-design replies the frozen guard caught 85.0%, below its development score.

In deployment a guard's rejections become template fallbacks: 38.7% of gpt-oss-20b calls, 33.3% of gpt-oss-120b and 63.3% of ALLaM-2 calls ended in the template.

G. Qualitative Over-Claims

43 development replies (14.3%), all in the automated band, asserted that a flood "is confirmed" or that water "has breached", although the state holds only a probability. No numeric check is designed to see them: the deployed guard passed 27 of the 43, and the judge 20. The wording comes partly from our own prompt, whose automated-band rule says "flood occurred", and the deployed template says "Flood has occurred", so the fallback itself over-claims. The frozen claim check flagged 8.0%, 17.6% and 5.1% of the replies to the three prompts of Table III, which it had never seen; how many of these flags are correct, and how many over-claims it misses, has not been measured. Our grounding claim is therefore limited to numbers.

H. The Disagreement Gate

On the 910 held-out events of the classifiers' split, the deployed gate held 26.9% of events and caught 91 of the policy model's 226 errors (40.3%); the straddle term alone caught 15.9%. We compared the gate with a single isotonic-calibrated XGBoost model without a gate, whose threshold is set on training folds for each cost ratio. With a false dispatch costing 1, a review 0.2 and the officer assumed always right, the gate was cheaper than no gate at every ratio from 1 to 10, and cheaper than the calibrated model while a missed dispatch costs less than 3.25 false dispatches (95% interval 2.25–4.60). The gate catches errors, but its net benefit depends on cost ratios that must be set operationally.

VI. DISCUSSION AND LIMITATIONS

All four faults we found lay at interfaces between components. A dispatch step bypassed a verified policy, a gate

tolerance was never related to the calibration of the models it compares, an untyped guard treated every number as interchangeable, and the template meant to absorb failures carried its own over-claim. A guard tuned on one model's errors met a different mix on the next. A fail-closed fallback masks the failures it absorbs unless its rate is monitored: while we prepared this study, retired API models and a token budget too small for reasoning models silently sent every reply to the template, which only the fallback rate revealed.

All rates come from one prompt family, 150 states, temperature 0.85 and three open-weight models from two families; no frontier model was tested. Reference labels were assigned by one author, who saw the checkers' verdicts when reading disagreements; replies on which all checks agreed were not read, and the claim check has no measured precision or recall. Blind labelling sheets for both gaps are prepared, and their scores are the main open item. The error categories come from keyword rules, not from reading. River and barrage values were sampled from operating ranges, not recorded events. The decision-path lemma covers extracted functions, not the asynchronous pipeline. The gate's costs are assumptions, and no officer has used the system in an exercise.

For a deployment we draw four rules. Prefer a narrator that does not write numbers, or failing that forbid the quantities the state lacks. Speak of probabilities in prompts and templates, and check assertion words as well as numbers. Monitor the template-fallback rate per model. And re-run a labelled held-out test whenever the model behind the API changes.

VII. Conclusion

In a flood-response pipeline where deterministic rules decide and an LLM narrates, 36.3% of live replies of two open-weight models stated an unsupported number, nearly all invented arrival times; the deployed guard caught 89.0% of them, and its misses on digit arrival times are close to the number that (4) predicts. A narrator that writes only placeholders is grounded by construction within a stated lexicon, and its one flagged reply in 229 fits the Proposition's second gap. A revised guard frozen before a third model family was queried met different errors there and was not significantly better. Qualitative over-claims, partly seeded by our own prompt and template, escape every numeric check. Our results support LLM narration only outside the decision path, with both numeric and claim checks against the state.

Acknowledgment

The authors thank the INDOFLOODS dataset authors and Open-Meteo for openly available data. In line with the IEEE policy on AI-generated content, the authors disclose that Claude (Anthropic) was used to help edit and restructure the text of all sections and to draft the experiment and plotting scripts. All experiments use the authors' own software and can be reproduced with fixed seeds; the live LLM replies are stored with the scripts. The authors checked every number, claim and reference and take full responsibility for the content.

References

[1] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023, doi: 10.1145/3571730.

[2] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[3] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. Conf. Lang. Model. (COLM)*, 2024.

[4] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[5] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. Cambridge, MA, USA: MIT Press, 2011.

[6] OASIS, "Common Alerting Protocol version 1.2," OASIS Standard, Jul. 2010.

[7] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, Art. no. 186345, 2024, doi: 10.1007/s11704-024-40231-1.

[8] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst. 33*, 2020, pp. 9459–9474.

[9] S. Min *et al.*, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 12076–12100, doi: 10.18653/v1/2023.emnlp-main.741.

[10] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 9004–9017, doi: 10.18653/v1/2023.emnlp-main.557.

[11] S. Dhuliawala *et al.*, "Chain-of-verification reduces hallucination in large language models," in *Findings Assoc. Comput. Linguist. (ACL)*, 2024, pp. 3563–3578, doi: 10.18653/v1/2024.findings-acl.212.

[12] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Adv. Neural Inf. Process. Syst. 36*, 2023, pp. 46595–46623, doi: 10.52202/075280-2020.

[13] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. Conf. Empir. Methods Nat. Lang. Process.: Syst. Demonstrations*, 2023, pp. 431–445, doi: 10.18653/v1/2023.emnlp-demo.40.

[14] Y. Zhang, S. Y. Emma, A. L. J. En, and J. S. Dong, "RvLLM: LLM runtime verification with domain knowledge," in *Adv. Neural Inf. Process. Syst. 38*, 2025, pp. 89374–89394, doi: 10.52202/085713-2693.

[15] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[16] S. Wiseman, S. Shieber, and A. Rush, "Challenges in data-to-document generation," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2017, pp. 2253–2263, doi: 10.18653/v1/D17-1239.

[17] C. Thomson and E. Reiter, "A gold standard methodology for evaluating accuracy in data-to-text systems," in *Proc. 13th Int. Conf. Nat. Lang. Gener. (INLG)*, 2020, pp. 158–168, doi: 10.18653/v1/2020.inlg-1.22.

[18] Z. Kasner and O. Dušek, "Beyond traditional benchmarks: Analyzing behaviors of open LLMs on data-to-text generation," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2024, pp. 12045–12072, doi: 10.18653/v1/2024.acl-long.651.

[19] T.-H. Wen, M. Gašić, N. Mrkšić, P.-H. Su, D. Vandyke, and S. Young, "Semantically conditioned LSTM-based natural language generation for spoken dialogue systems," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2015, pp. 1711–1721, doi: 10.18653/v1/D15-1199.

[20] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 2669–2678, doi: 10.1609/aaai.v32i1.11797.

[21] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Adv. Neural Inf. Process. Syst. 30*, 2017, pp. 6402–6413.

[22] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[23] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.